

Guarded Block Transfers for Polytope Distance: Linear Convergence with Explicit Constants and Gap-Certified Screening for SVM Training

Mayank Gupta

Independent Researcher

IAMMAYANKG@GMAIL.COM

Abstract

Hard-margin SVM training is the distance problem between two convex hulls, which the Triangle Algorithm (TA) solves with a certified duality gap. We propose *guarded block transfers*: k Mitchell–Demjanov–Malozemov (MDM) transfers per scan are aggregated into one exact line search, guarded by a comparison with the single best transfer at a cost independent of the number of points n , and replaced by an away step when the leading transfer is capacity-clipped. On the Minkowski-difference polytope, where the objective is 1-strongly convex, we prove global linear convergence with explicit constants and no combinatorial swap-step bounds; the per-step contraction is k -independent and only the drop-count bound grows with k , so aggregation is provably safe while the saving in scans is measured, not proved. Since the TA certificate is the Frank–Wolfe gap divided by the current distance, gap-safe screening applies, and we bound the iteration after which it has removed every zero-weight point off the optimal faces by $O\left(\frac{k}{\rho} \log \frac{1}{\tau}\right)$ up to logarithmic factors in the geometry, ρ the rate constant and τ the optimal-face margin. On five LIBSVM benchmarks the solver matches the baselines to five digits wherever both converge, ties LIBSVM on sparse-support hard margins, beats BPCG and kernel SMO on kernel L_2 margins (where LIBSVM on a precomputed kernel is faster still), and is two to three orders of magnitude slower on dense-support L_2 margins and nearly touching kernel hulls, where LIBLINEAR finishes in seconds. We report both sides.

1. Introduction

Training a linear hard-margin SVM is equivalent to finding the closest points of the two classes’ convex hulls: the optimal separator is the perpendicular bisector of the witness pair and the margin is half the hull distance [8]. Kalantari’s Triangle Algorithm (TA) exploits this *distance duality* directly [1]: TA I decides whether the hulls intersect, and TA II computes their distance while maintaining an iterate pair (p, q) and a certified interval $[LB, UB]$ from parallel supporting hyperplanes. The enhanced TA (ETA) of [2] adds dot-product caching and search heuristics. Below, ETA denotes that solver with our guarded block transfers, K-ETA its kernelised form, and “our SMO” our own SMO implementation.

The workhorse step of such solvers is the *weight transfer* from the worst active vertex to the best candidate, the Mitchell–Demjanov–Malozemov (MDM) step [4], which is the pairwise Frank–Wolfe (FW) step of [3] on the relevant polytope. Each step costs an $O(n)$ scan and yields one transfer; we ask how much of the scan can be reused across several transfers without weakening the guarantee. Our contributions:

1. **Guarded block transfers** (§3): pair the top- k receivers with the worst- k active donors, take one exact line search along the aggregated direction, *guard* it by an $O(k^2)$ comparison with the single MDM step, and fall back to an away step on capacity-clipped iterations.
2. **Linear convergence with explicit constants** (Theorem 3): on $Z = \text{conv}(V) - \text{conv}(W)$ the guarded block algorithm inherits the pyramidal-width rate of pairwise FW up to a constant, and the away fallback replaces the combinatorial swap-step bound by linear drop-step counting (cf. [12]).
3. **Screening activation with explicit constants** (Theorem 6): the TA certificate is the FW gap divided by the distance (Lemma 5), so gap-safe screening [17] applies, and we bound the iteration by which it has removed every zero-weight point off the optimal faces.
4. **A regime study on real data** (§5): two batteries on five LIBSVM datasets locate the boundary from both sides.

Code, proofs, numerical checks and experiment scripts are at <https://github.com/iammayankg/triangle-algorithm-sv>.

2. Setting

Let $V = \{v_1, \dots, v_n\}$, $W = \{w_1, \dots, w_m\} \subset \mathbb{R}^d$. The polytope distance problem is

$$\min_{p \in \text{conv}(V), q \in \text{conv}(W)} \frac{1}{2} \|p - q\|^2. \quad (1)$$

Writing $z = p - q$ and $Z := \text{conv}(V) - \text{conv}(W) = \text{conv}\{v_i - w_j\}$, (1) is the minimum-norm-point problem $\min_{z \in Z} F(z)$, $F(z) = \frac{1}{2} \|z\|^2$, whose objective is 1-strongly convex and 1-smooth. Let z^* be the minimiser, $\delta^* = \|z^*\|$, $h(z) = F(z) - F(z^*)$, $M = \text{diam}(Z)$ and $\delta = \text{PW}(A) > 0$ the pyramidal width [3] of the atom set $A = \{v_i - w_j\}$ (a property of the atom set, not of Z alone).

The algorithm keeps simplex weights α on V , β on W (iterate z_t after t iterations), with $p = \sum \alpha_i v_i$, $q = \sum \beta_j w_j$; the induced product representation $\lambda_{ij} = \alpha_i \beta_j$ of z over the atoms $v_i - w_j$ has support $\text{supp } \alpha \times \text{supp } \beta$. With $g = -\nabla F(z) = q - p$, the *side scores* are $s_i^V = \langle g, v_i \rangle$ and $s_j^W = \langle p - q, w_j \rangle$, maintained incrementally from cached arrays.

Certificate. For any iterate, $\text{UB} = \|z\|$ and $\text{LB} = \min_{u \in Z} \langle z, u \rangle / \|z\| = (\min_i \langle z, v_i \rangle - \max_j \langle z, w_j \rangle) / \text{UB}$ satisfy $\text{LB} \leq \delta^* \leq \text{UB}$; the TA stops when the certified relative gap $(\text{UB} - \text{LB}) / \text{UB}$ is at most ε . The L_2 soft margin with regularisation parameter C , the ν -SVM and the kernel variants reduce to (1) exactly (augmented coordinates that turn the Gram matrix K into $K + I/C$; reduced convex hulls; feature-space images), see Appendix B.

Gaps. $g^{\text{FW}} = \max_{u \in Z} \langle g, u \rangle - \langle g, z \rangle$, $g^{\text{A}} = \langle g, z \rangle - \min_{u \in \text{supp } \lambda} \langle g, u \rangle$, $g^{\text{PW}} = g^{\text{FW}} + g^{\text{A}}$. Because $\langle g, v_i - w_j \rangle = s_i^V + s_j^W$ is separable and the support is a product set, each gap splits into a V -side and a W -side term computable from the score arrays in $O(n + m)$ (Lemma 7), written $g_V^{\text{PW}}, g_W^{\text{PW}}$ (likewise for FW and A), with g_{side} the term of the side being stepped; a transfer $v_r - v_u$ on the V side is a pairwise direction of Z (Lemma 8).

Algorithm 1: One iteration of the guarded block-transfer step (side V ; W symmetric)

top- k receivers by s^V ; worst- k donors in $\text{supp } \alpha$; pair best-with-worst: (r_j, u_j) , $j \leq k'$ ($k' \leq k$ pairs, limited by the support size); pair 1 is the MDM pair (r^*, u^*)
 $d_j \leftarrow v_{r_j} - v_{u_j}$; $\text{gap}_j \leftarrow s_{r_j}^V - s_{u_j}^V$; discard pairs with $\text{gap}_j \leq 0$ and receivers that are donors (if r^* is a donor it leaves the donor list, so pair 1 survives); $\gamma_j \leftarrow \min(\text{gap}_j / \|d_j\|^2, c_j)$
 // $c_j = \alpha_{u_j}$: donor capacity
 1 **if** pair 1 uncapped ($\gamma_1 = \text{gap}_1 / \|d_1\|^2$) [case (a)] **then**
 2 $\Delta_B \leftarrow$ gain of exact line search $\theta \in [0, 1]$ along $d = \sum_j \gamma_j d_j$; $\Delta_1 \leftarrow$ gain of the MDM step
 3 take the step with the larger gain // guard, $O(k'^2)$
 4 **else**
 5 **if** the away step on u^* is boundary-clipped [case (b)] **then**
 6 take it // drop step: $\alpha_{u^*} \rightarrow 0$
 7 **else**
 8 take the best of {block, MDM, away(u^*), toward(r^*)}
 9 **end**
 10 **end**

3. Guarded block transfers

Algorithm 1 amortises one $O(n)$ scan over up to k transfers. Only the scan is amortised: a new receiver still costs one Gram or kernel column, $O(nd)$. Every candidate gain is available from cached quantities at a cost independent of n : with $S = \langle g, d \rangle = \sum_j \gamma_j \text{gap}_j$ and $D = \|d\|^2 = \gamma^\top \Gamma \gamma$, Γ the $k' \times k'$ Gram matrix of the pair directions ($O(k'^2)$ cached dot products), the block gain is $\theta S - \theta^2 D / 2$ at $\theta = \min(S/D, 1)$; the MDM, away and toward gains are the usual one-dimensional expressions. The *gain* of a step is the decrease $h(z_t) - h(z_{t+1})$. The cache update for a block is one matrix–vector product with the net per-index coefficients; columns are computed on first use and cached, so support columns cost nothing after their first appearance. Feasibility is immediate, since donors lose at most $\theta \gamma_j \leq c_j$. We call an iteration *case (a)* if pair 1 is uncapped and *case (b)* otherwise; the away step of line 6, which drives α_{u^*} to zero, is a *drop* step (a capped block pair may also zero a donor, which only helps the counts below).

Safe screening. With $r = \sqrt{\text{UB}^2 - \max(\text{LB}, 0)^2}$ and $v_{\min} = \arg \min_i \langle z, v_i \rangle$, a zero-weight point with $\langle z, v_i \rangle - \langle z, v_{\min} \rangle > r \|v_i - v_{\min}\|$ is outside the optimal support and is removed (symmetrically on W with the max). Because the optimum’s support survives, the reduced problem has the same optimum and all subsequent lower bounds remain valid.

4. Analysis

Full proofs and the conventions for degenerate cases are in Appendix A.

Lemma 1 (Block gain) *Let $b_j = \gamma_j \|d_j\|$ over the k' retained pairs, all with $\text{gap}_j > 0$. Then $S \geq \sum_j b_j^2$, $D \leq k' S$, and the realised block gain satisfies $\Delta_B \geq S / (2k') \geq \Delta_1 / (2k')$, Δ_1 the MDM gain.*

With the guard, every case-(a) iteration gains $\max(\Delta_B, \Delta_1) \geq \Delta_1$: the guarded step never makes less progress than pairwise FW.

Lemma 2 (Good-step progress) *If the iteration works on the side with the larger pairwise gap, then on any non-drop iteration gain $\geq \min((g^{\text{PW}})^2/(32M^2), g^{\text{PW}}/8)$.*

Theorem 3 (Global linear convergence) *Suppose the guarded block-transfer algorithm with away fallback is run for one side per iteration (the side with the larger pairwise gap), using any $k \geq 1$, from any vertex pair (initial support $s_0 = 2$). Let $h_t = h(z_t)$. Then every non-drop iteration satisfies $h_{t+1} \leq (1 - \rho)h_t$ with*

$$\rho = \frac{1}{16} (\delta/M)^2.$$

For any horizon T , among the first T iterations the number of drop steps is at most $kT_{\text{good}} + s_0$, where T_{good} is the number of non-drop iterations; consequently, $h_T \leq h_0 (1 - \rho)^{(T-s_0)/(k+1)}$. Under the two-sided schedule of the implementation (larger-gap side first, second side skipped after a drop) every non-drop iteration has the same contraction, the drop count is at most $2kT_{\text{good}} + s_0$, and $h_T \leq h_0 (1 - \rho)^{(T-s_0)/(2k+1)}$ (Lemma 10).

The rate combines Lemma 2 with $g^{\text{PW}} \geq \delta\sqrt{2h}$ and $g^{\text{PW}} \geq g^{\text{FW}} \geq h$ (Lemma 9). Drop counting is the away-step argument: each drop removes one support index and non-drop steps add at most k . No swap-step bound is needed: a case-(b) iteration either drops (counted above) or takes the best of four candidates, of which the toward or away step alone gains Lemma 2's amount, whichever wins.

Proposition 4 (Near-orthogonal speed-up) *If all k' pairs are uncapped and $|\langle d_i, d_j \rangle| \leq \eta \|d_i\| \|d_j\|$ for $i \neq j$, then the block gain is at least $(\sum_j \Delta_j)/(1 + \eta(k' - 1))$, where Δ_j is pair j 's individual gain.*

As $\eta \rightarrow 0$ one scan realises the sum of all k pairwise gains.

Lemma 5 (The certified gap is the FW gap) *At any full scan, with LB evaluated at the current $z \neq 0$, $\text{UB} - \text{LB} = g^{\text{FW}}(z)/\text{UB}$ (a retained best lower bound gives $\leq$), and $g^{\text{FW}}(z) \leq 2h + R\sqrt{2h}$ with $R = \max_{u \in Z} \|u\|$, the radius of Z about the origin.*

Theorem 6 (Screening activation) *Assume $\delta^* > 0$. Let $F_V = \{i : \langle z^*, v_i \rangle = \min_j \langle z^*, v_j \rangle\}$ and, with the max, F_W be the optimal faces: the vertices of zero score margin, which contain the optimal supports (possibly strictly). Let $\tau_i = \langle z^*, v_i \rangle - \min_j \langle z^*, v_j \rangle$ be the score margin of v_i (and symmetrically on W), with $\tau > 0$ the minimal nonzero margin. Finally, let L be the larger class diameter.*

*A zero-weight point with margin τ_i is screened as soon as $r < \tau_i/(3L)$. Consequently, once $h_t < H := \tau^4 \delta^{*2}/(2592 R^4 L^4)$, every zero-weight point of V outside F_V and of W outside F_W is removed permanently at the next screening round. Under Theorem 3 this holds for every iteration $t > s_0 + \frac{a_k}{\rho} \log \max\{1, h_0/H\}$, with $a_k = k + 1$ one-sided and $2k + 1$ two-sided. Following this iteration, each subsequent scan costs $O(|F_V| + |F_W| + |\text{supp } \alpha| + |\text{supp } \beta|)$ instead of $O(n + m)$.*

In words: since $r^2 \leq 2R(\text{UB} - \text{LB})$, a point with margin τ_i is screened once the certified gap falls below $\tau_i^2/(18RL^2)$. The faces can strictly contain the supports: for $V = \{(1, 0), (1, 1), (2, 0)\}$ and $W = \{0\}$, $z^* = (1, 0)$ and $(1, 1)$ has zero weight and zero margin, so no gap-safe rule can remove it. Lemmas 1, 5 and 10, Proposition 4 and Theorem 6 are verified numerically on random and adversarial ensembles at machine precision (Appendix E).

Table 1: Regime battery on real data, five seeds, 600 s per solver, mean $\pm$ 95% CI. Kernel cells (RBF with width parameter $1/d$) use a class-balanced subsample of n_k points. Same objective per row: LIN, linear hard margin (ETA with screening vs LIBSVM’s $C=10^6$ surrogate; our SMO in brackets); KHM, feature-space hard margin (K-ETA vs the same surrogate); KL2, kernel L_2 margin, $C=1$ (K-ETA vs kernel SMO, exact objective; in brackets LIBSVM on the precomputed $K + I/C$ kernel, fit time + kernel construction). conv = seeds in which ETA certified; oracle = $O(nd)$ column (ETA) or row (SMO) evaluations, thousands; SMO’s include rows recomputed after eviction from its 512-row cache. Run with the released code (fallback in every path, schedule of Lemma 10).

cell	dataset	n_k / d	ETA (s)	baseline (s)	conv	dist. ETA / base	acc. ETA / base	oracle
LIN	gisette	6000 / 5000	31.6 ± 2.8	36.5 ± 5.9 [259 $\pm$ 31]	5/5	3.6638 / 3.6640	.981 / .981	1.5 [11.1]
KHM	gisette	6000 / 5000	57.8 ± 8.7	68.1 ± 13.4	5/5	.05215 / .05215	.979 / .979	2.6
KHM	ijcnn1	9853 / 22	351 ± 109	1.1 ± 0.2	5/5	.00430 / .00430	.946 / .946	2.6
KHM	a9a	10000 / 123	> 600	19.8 ± 1.3	0/5	$\leq .00006$ / .0033	.707 / .752	5.9
KHM	covtype	10000 / 54	> 600	134 ± 3 (maxiter)	0/5	$\leq .00055$ / .00025	.694 / .712	7.5
KHM	w8a	6479 / 300	11.7 ± 32.4	—	5/5	hulls intersect	—	0.5
KL2	gisette	6000 / 5000	84.8 ± 15.4	243 ± 15 [$0.5 \pm 0.1 + 3$]	5/5	.06882 / .06882	.971 / .971	3.7 / 15.2
KL2	w8a	6479 / 300	8.7 ± 1.6	28.5 ± 1.3 [$0.6 \pm 0.1 + 1$]	5/5	.05682 / .05682	.975 / .975	3.6 / 20.2
KL2	a9a	10000 / 123	15.3 ± 2.8	27.0 ± 4.7 [$2.2 \pm 0.2 + 2$]	5/5	.03019 / .03019	.795 / .795	7.6 / 43.6
KL2	covtype	10000 / 54	21.4 ± 1.8	15.4 ± 0.6 [$2.1 \pm 0.1 + 2$]	5/5	.02609 / .02609	.792 / .792	9.1 / 56.2
KL2	ijcnn1	9853 / 22	7.7 ± 1.9	5.3 ± 0.4 [$1.1 \pm 0.1 + 2$]	5/5	.04790 / .04790	.959 / .959	4.6 / 26.1

5. Experiments

Setup. All solvers are validated against an exact QP (distances agree to 10^{-9} or better). Stopping rule $\varepsilon = 10^{-3}$ on the certified relative gap unless stated; baselines use matched tolerances on the same objective, LIBLINEAR up to its intercept penalty (Appendix C). Synthetic instances follow [2] (two Gaussian clouds, $n = 5,000$ per class). Real data are the LIBSVM sets a9a, w8a, ijcn1, gisette and covtype (100k subsample), standardised, official test sets where they exist, five seeds, 600 s per solver and cell, all on one dedicated 32-vCPU Xeon 2.6 GHz machine (Appendix C).

Blocks and screening (Appendix C). In isolated runs (Table 6) the guard and fallback alone (guarded $k=1$) cut plain pairwise FW’s iterations 1.4–2 $\times$ but its time by at most 12%; aggregation cuts iterations a further 4.2–4.4 $\times$, which buys wall-clock only where per-iteration $O(n)$ work, not column computation, dominates: 1.9 $\times$ over guarded $k=1$ on ijcn1 ($d=22$; 3.0 $\times$ over BPCG), nothing on gisette, where columns take 86–92% of the time. Blocks cut iterations more on synthetic data (7–19 $\times$ over MDM at $k=16$) than on real data (6–9 $\times$) because the near-orthogonality of Proposition 4 fails on real features (measured pair correlation 0.07–0.29) and drops take up to 36% of real iterations; a cost-model choice of k is in Appendix C. Screening gives 1.1–2.9 $\times$ at $\varepsilon = 10^{-5}$ and is neutral at 10^{-3} .

Real data, sparse-support regime (Table 1). TA I first certifies separability (gisette yes, the other four no, in 0.1–35 s). On gisette ($d=5,000$; 1,237 of 6,000 points in the support) ETA ties LIBSVM on the linear hard margin, both 7–8 $\times$ ahead of our SMO; K-ETA is 1.2 $\times$ faster than LIBSVM on the RBF hard margin and 2.9 $\times$ faster than kernel SMO on the kernel L_2 margin, with identical solutions. Across all five KL2 cells K-ETA matches kernel SMO’s solution with 4–6 $\times$ fewer kernel-column

evaluations, partly a cache effect (our SMO evicts from a 512-row cache, ETA never evicts); in wall-clock it is $3\times$ faster where columns are expensive (gisette, w8a) and $1.4\times$ slower where they are cheap (ijcnn1, covtype). Against BPCG [12] with the same column cache and certificate (Table 7), block ETA needs $12\text{--}35\times$ fewer iterations, is $2.5\text{--}5.6\times$ faster on the five KL2 cells and ties on the gisette L_2 margin, where columns dominate. LIBSVM on the precomputed $K + I/C$ kernel (brackets in Table 1) reaches the same distances $2.5\text{--}24\times$ faster than K-ETA, kernel construction included: the KL2 wins are over column-oracle solvers only.

The KHM column shows where the geometry turns against the method. On ijcnn1 both return the same solution (819 support vectors), but K-ETA needs $1.6\text{--}2.7 \times 10^5$ iterations and 351 s against LIBSVM’s 1.1 s: the hulls are 0.004 apart in a space of diameter $\approx \sqrt{2}$, so a relative certificate needs an absolute gap of order 10^{-6} , which suits LIBSVM’s second-order selection far better. On a9a and covtype the hulls are within 10^{-4} and 5×10^{-4} of touching, and neither solver certifies anything. On w8a the hulls intersect (duplicate points, opposite labels); K-ETA certifies this in 12 s.

Real data, L_2 soft margins (Table 2, appendix). The same boundary appears within one problem class. Gisette is separable, so its L_2 margin has a sparse support and ETA converges to the certified gap in 28–30 s at every C , $7\times$ faster than our SMO. LIBLINEAR at matched tolerance (10^{-6}) takes 8 min to 1.4 h; at its default 10^{-4} it takes 3.5–51 s but stops 5–24% above the certified optimum with lower test accuracy (Table 4). On gisette ETA’s advantage over LIBLINEAR is the certificate, not wall-clock. On the four overlapping datasets LIBLINEAR finishes in 0.1–23 s, while the geometric solvers converge within the cap only on ijcnn1 at $C=0.1$ (ETA 418 s, our SMO 200 s); at $C=10$ the capped ETA iterate loses 2–34 accuracy points, and on covtype it is not usable at any C . Uncapped, ETA certifies 10^{-3} on a9a, w8a and ijcnn1 at $C=0.1$ in 349–1,107 s, $80\text{--}2,200\times$ LIBLINEAR’s time (Figure 1, Table 5), at a rate at least linear but with $O(n^2)$ -type cost on these dense supports (heuristic in the caption of Figure 1).

6. Related work

Distance duality and the TA [1, 2]; MDM, Gilbert and Wolfe [4, 5, 25, 26]; FW for polytope distance and SVMs [6, 8, 21–24]; away steps, pyramidal width and facial distance [3, 7, 27]. Boosted FW [13], which also aggregates atoms into one direction, swap-step-free pairwise variants [12, 14] and block-coordinate FW [9] are the closest algorithmic relatives; gap-safe screening [15, 17, 18] and active-set identification [19, 20] the closest to Theorem 6, with [10, 11, 16] as its SVM ancestors (Appendix D).

7. Conclusion

Guarded block transfers give the Triangle Algorithm a linear rate with explicit constants and no combinatorial swap-step bounds, and a safe screening rule with a finite activation bound. The experiments locate the boundary: on sparse supports and well-separated hulls the solver ties LIBSVM, beats the column-oracle solvers BPCG and kernel SMO (not LIBSVM with the kernel in memory) and supplies a certificate; on dense supports or nearly touching hulls LIBLINEAR or LIBSVM is two to three orders of magnitude faster. Limitations: screening pays only when the certificate closes; the code is NumPy-level and dense; the constants are loose; the ν -SVM variant is unanalysed; k is fixed (Appendix C gives a cost-model choice).

References

- [1] B. Kalantari. A characterization theorem and an algorithm for a convex hull problem. *Annals of Operations Research*, 226(1):301–349, 2015.
- [2] M. Gupta and B. Kalantari. An enhanced triangle algorithm for large-scale support vector machine optimization. *Journal of Intelligent Decision Making and Information Science*, 3(9s), 2026.
- [3] S. Lacoste-Julien and M. Jaggi. On the global linear convergence of Frank–Wolfe optimization variants. *NeurIPS*, 2015.
- [4] B. F. Mitchell, V. F. Demyanov, and V. N. Malozemov. Finding the point of a polyhedron closest to the origin. *SIAM J. Control*, 12(1):19–26, 1974.
- [5] E. G. Gilbert. An iterative procedure for computing the minimum of a quadratic form on a convex set. *SIAM J. Control*, 4(1):61–80, 1966.
- [6] S. S. Keerthi, S. K. Shevade, C. Bhattacharyya, and K. R. K. Murthy. A fast iterative nearest point algorithm for support vector machine classifier design. *IEEE Trans. Neural Networks*, 11(1):124–136, 2000.
- [7] J. Guélat and P. Marcotte. Some comments on Wolfe’s ‘away step’. *Mathematical Programming*, 35(1):110–119, 1986.
- [8] K. P. Bennett and E. J. Bredensteiner. Duality and geometry in SVM classifiers. *ICML*, 2000.
- [9] S. Lacoste-Julien, M. Jaggi, M. Schmidt, and P. Pletscher. Block-coordinate Frank–Wolfe optimization for structural SVMs. *ICML*, 2013.
- [10] L. El Ghaoui, V. Viallon, and T. Rabbani. Safe feature elimination in sparse supervised learning. *Pacific J. Optimization*, 8(4):667–698, 2012.
- [11] K. Ogawa, Y. Suzuki, and I. Takeuchi. Safe screening of non-support vectors in pathwise SVM computation. *ICML*, 2013.
- [12] K. Tsuji, K. Tanaka, and S. Pokutta. Pairwise conditional gradients without swap steps and sparser kernel herding. *ICML*, 2022.
- [13] C. W. Combettes and S. Pokutta. Boosting Frank–Wolfe by chasing gradients. *ICML*, 2020.
- [14] F. Rinaldi and D. Zeffiro. Avoiding bad steps in Frank–Wolfe variants. *Computational Optimization and Applications*, 84(1):225–264, 2023.
- [15] O. Fercoq, A. Gramfort, and J. Salmon. Mind the duality gap: safer rules for the Lasso. *ICML*, 2015.
- [16] J. Wang, P. Wonka, and J. Ye. Scaling SVM and least absolute deviations via exact data reduction. *ICML*, 2014.
- [17] E. Ndiaye, O. Fercoq, A. Gramfort, and J. Salmon. Gap safe screening rules for sparsity enforcing penalties. *JMLR*, 18(128):1–33, 2017.

- [18] A. Shibagaki, M. Karasuyama, K. Hatano, and I. Takeuchi. Simultaneous safe screening of features and samples in doubly sparse modeling. *ICML*, 2016.
- [19] I. M. Bomze, F. Rinaldi, and D. Zeffiro. Active set complexity of the away-step Frank–Wolfe algorithm. *SIAM J. Optim.*, 30(3):2470–2500, 2020.
- [20] D. Garber. Revisiting Frank–Wolfe for polytopes: strict complementarity and sparsity. *NeurIPS*, 2020.
- [21] B. Gärtner and M. Jaggi. Coresets for polytope distance. *SoCG*, 2009.
- [22] K. L. Clarkson. Coresets, sparse greedy approximation, and the Frank–Wolfe algorithm. *ACM Trans. Algorithms*, 6(4):63, 2010.
- [23] I. W. Tsang, J. T. Kwok, and P.-M. Cheung. Core vector machines: fast SVM training on very large data sets. *JMLR*, 6:363–392, 2005.
- [24] R. Nanculef, E. Frandi, C. Sartori, and H. Allende. A novel Frank–Wolfe algorithm. Analysis and applications to large-scale SVM training. *Information Sciences*, 285:66–99, 2014.
- [25] J. López and J. R. Dorronsoro. Linear convergence rate for the MDM algorithm for the nearest point problem. *Pattern Recognition*, 48(4):1510–1522, 2015.
- [26] P. Wolfe. Finding the nearest point in a polytope. *Mathematical Programming*, 11(1):128–149, 1976.
- [27] J. Peña and D. Rodríguez. Polytope conditioning and linear convergence of the Frank–Wolfe algorithm. *Mathematics of Operations Research*, 44(1):1–18, 2019.

Acknowledgements. We thank the OPT 2026 reviewers; one of them supplied the example after Theorem 6.

AI assistance. AI tools were used in the preparation of the code, experiments and manuscript; the author verified all results and is responsible for the content.

Appendix A. Proofs

Conventions. $M > 0$ (a singleton Z is solved at initialisation), and the algorithm stops with “intersect” when $UB = \|z\| = 0$, before any division by UB . Lemma 1 and Algorithm 1 presuppose $k' \geq 1$: a side with no positive-gap pair is skipped, and on a non-drop iteration the side with the larger pairwise gap has one. An away direction whose donor has weight one is unavailable (the iterate is that vertex and the away gap of that side is zero). In Proposition 4, $\eta \geq 0$.

Lemma 7 (Separability) *With the product representation, $g^{\text{PW}} = g_V^{\text{PW}} + g_W^{\text{PW}}$ where $g_V^{\text{PW}} = \max_i s_i^V - \min_{i \in \text{supp } \alpha} s_i^V \geq 0$ and g_W^{PW} likewise; the same holds for g^{FW} and g^{A} .*

Proof. $\langle g, v_i - w_j \rangle = s_i^V + s_j^W$ is separable and the support is a product set, so the max over atoms and the min over the support separate. Each side term is nonnegative: the FW term is the maximum score minus the weighted mean of the support scores, the away term is that mean minus the support minimum, and a weighted mean lies between the minimum and the maximum. ■

Lemma 8 (Side transfers are Z -pairwise directions) *For $\alpha_u > 0$ and any $b \in \text{supp } \beta$, $v_r - v_u = (v_r - w_b) - (v_u - w_b)$ is a difference of an atom of Z and an atom in the support, and the transfer $\gamma \leq \alpha_u$ realises $z \leftarrow z + \gamma(v_r - v_u)$ inside Z . The side capacity $\alpha_u = \sum_b \alpha_u \beta_b$ is the total weight of the product atoms $v_u - w_b$, $b \in \text{supp } \beta$: the side transfer is the sum of the atom-pair transfers of masses $\gamma \beta_b$ from $v_u - w_b$ to $v_r - w_b$, and for $r \neq u$ this weight update is feasible exactly when $0 \leq \gamma \leq \alpha_u$; hence every such transfer remains in Z .*

Proof of Lemma 1. Since $\gamma_j \leq \text{gap}_j / \|d_j\|^2$, we have $\gamma_j \text{gap}_j \geq \gamma_j^2 \|d_j\|^2 = b_j^2$. Consequently, $S = \sum_j \gamma_j \text{gap}_j \geq \sum_j b_j^2$. By the triangle inequality, $\|d\| \leq \sum_j \gamma_j \|d_j\| = \sum_j b_j$, and by Cauchy-Schwarz, $(\sum_j b_j)^2 \leq k' \sum_j b_j^2 \leq k' S$; hence $D \leq k' S$. Exact line search on the 1-smooth F gives $\Delta_B = S^2 / (2D) \geq S / (2k')$ in the interior case and $\Delta_B = S - D/2 \geq S/2$ at the boundary $\theta = 1$, where $D \leq S$. Finally, $S \geq \gamma_1 \text{gap}_1 \geq \Delta_1$, since all terms are nonnegative and $\Delta_1 = \gamma_1 \text{gap}_1 - \gamma_1^2 \|d_1\|^2 / 2$. ■

Proof of Lemma 2. Let $g_S = \max(g_V^{\text{PW}}, g_W^{\text{PW}}) \geq g^{\text{PW}} / 2$. Every direction below is a difference of two points of one class hull, so its norm is at most $\text{diam}(\text{conv } V) \leq M$. For a feasible direction with directional derivative $\sigma > 0$, squared norm D and maximal step $\bar{\theta}$, exact line search on a 1-smooth function gives gain $\geq \min(\sigma^2 / (2D), \bar{\theta} \sigma / 2)$ (interior optimum, or the clipped step evaluated with $\bar{\theta} D \leq \sigma$). In case (a), the guard's gain is at least the MDM gain, whose derivative is g_S and whose interior optimum is attained, yielding gain $\geq g_S^2 / (2M^2) \geq (g^{\text{PW}})^2 / (8M^2)$. In case (b), non-drop, $g_S = g_{\text{side}}^{\text{FW}} + g_{\text{side}}^{\text{A}}$, so one term is at least $g_S / 2 \geq g^{\text{PW}} / 4$. If it is the FW term, the toward step (maximal step 1) gives gain $\geq \min((g_{\text{side}}^{\text{FW}})^2 / (2M^2), g_{\text{side}}^{\text{FW}} / 2) \geq \min((g^{\text{PW}})^2 / (32M^2), g^{\text{PW}} / 8)$; if it is the away term, the away line search is interior (non-drop) and gain $\geq (g_{\text{side}}^{\text{A}})^2 / (2M^2) \geq (g^{\text{PW}})^2 / (32M^2)$. The algorithm takes the best available candidate. ■

Lemma 9 (Geometric strong convexity; [3, Thm. 6] with $\mu = 1$) *For every $z \in Z$ with a representation whose support contains an away atom, $g^{\text{PW}}(z) \geq \delta \sqrt{2h(z)}$ and $g^{\text{PW}}(z) \geq g^{\text{FW}}(z) \geq h(z)$.*

Proof. This is Theorem 6 of [3] for a 1-strongly convex objective; we include the short argument. If $z = z^*$ both inequalities are immediate ($h = 0$ and all gaps are nonnegative). Otherwise let $e = (z^* - z) / \|z^* - z\|$; the pyramidal-width inequality in the proof of [3, Thm. 6] (Definition 3 there, applied to the direction g together with $\langle g, e \rangle \leq \|g\|$) gives, for the FW atom s and the away atom v , $\langle g, s - v \rangle \geq \delta \langle g, e \rangle$. By 1-strong convexity, $\langle g, z^* - z \rangle \geq h + \frac{1}{2} \|z^* - z\|^2$, so with $\varrho = \|z^* - z\|$, $g^{\text{PW}} \geq \delta(h/\varrho + \varrho/2) \geq \delta \sqrt{2h}$ by AM-GM. Finally $g^{\text{PW}} = g^{\text{FW}} + g^{\text{A}} \geq g^{\text{FW}} = \max_u \langle g, u - z \rangle \geq \langle g, z^* - z \rangle \geq h$ by convexity. ■

Proof of Theorem 3. By Lemmas 2 and 9 a non-drop iteration gains at least

$$\min(2\delta^2 h / (32M^2), h/8) = (\delta/M)^2 h / 16$$

using $\delta \leq M$; hence $h_{t+1} \leq (1 - \rho) h_t$. Drop counting: non-drop iterations add at most k support indices, drop iterations add none and remove at least one, and the support size stays ≥ 2 from s_0 ; so the number of removals, hence of drops, is at most $kT_{\text{good}} + s_0$, so that

$$T_{\text{good}} \geq (T - s_0) / (k + 1).$$

All steps, drops included, are descent steps, so h is monotone. $\blacksquare$

Lemma 10 (Two-sided schedule) *Let every iteration compute both side gaps, take the guarded step of Algorithm 1 on the side with the larger pairwise gap first, and take the other side's guarded step only if the first step was not a drop; an iteration counts as a drop iff its first step is a drop. Then every non-drop iteration gains at least $\min((g^{\text{PW}})^2/(32M^2), g^{\text{PW}}/8)$ and the number of drops is at most $2kT_{\text{good}} + s_0$; hence Theorem 3 holds for this schedule with the same ρ and exponent $(T - s_0)/(2k + 1)$.*

Proof. On a non-drop iteration the first step is exactly the step analysed in Lemma 2, so it gains at least the stated amount; the second step is an exact line search along a feasible descent direction (or is skipped), so it cannot increase h . Drop counting: a drop iteration takes a single away step that removes one support index and adds none, because the second side is skipped; a non-drop iteration takes two guarded steps and adds at most k indices each. Removals, hence drops, therefore number at most $2kT_{\text{good}} + s_0$. $\blacksquare$

Remark 11 *The skip is what makes the count work. If the second side always steps, an iteration whose analysed side drops can add up to k indices on the other side, and the count becomes $T_{\text{drop}} \leq 2kT_{\text{good}} + kT_{\text{drop}} + s_0$, which is vacuous for $k \geq 1$.*

Proof of Proposition 4. $D = \sum_{i,j} \gamma_i \gamma_j \langle d_i, d_j \rangle \leq \sum_j b_j^2 + \eta \sum_{i \neq j} b_i b_j \leq (1 + \eta(k' - 1)) \sum_j b_j^2 \leq (1 + \eta(k' - 1))S$ by Cauchy–Schwarz on the cross terms. If $D \geq S$ the line search is interior and $\Delta_B = S^2/(2D) \geq S/(2(1 + \eta(k' - 1)))$; if $D < S$ then $\theta = 1$ and $\Delta_B = S - D/2 > S/2$, which is larger. For uncapped pairs $S = \sum_j 2\Delta_j$. $\blacksquare$

Proof of Lemma 5. $\text{LB} = \min_{u \in Z} \langle z, u \rangle / \|z\|$ by definition of the supporting hyperplane bound and $\text{UB} = \|z\|$; subtracting, $\text{UB} - \text{LB} = (\langle z, z \rangle - \min_u \langle z, u \rangle) / \|z\| = g^{\text{FW}} / \text{UB}$. At the optimum $0 = \langle z^*, z^* \rangle - \min_u \langle z^*, u \rangle$; subtracting from $g^{\text{FW}}(z)$, $g^{\text{FW}}(z) = (\|z\|^2 - \|z^*\|^2) + (\min_u \langle z^*, u \rangle - \min_u \langle z, u \rangle) \leq 2h + \max_u \langle z^* - z, u \rangle \leq 2h + R\|z - z^*\| \leq 2h + R\sqrt{2h}$ by 1-strong convexity. $\blacksquare$

Proof of Theorem 6. Strong convexity gives $\|z - z^*\| \leq r$ for the screening radius $r = \sqrt{\text{UB}^2 - \max(\text{LB}, 0)^2}$ (τ is defined only if some point has positive margin; otherwise there is nothing to screen). Let $m^* = \min_j \langle z^*, v_j \rangle$, choose $j' \in F_V$ and set $v' = v_{j'}$; since $v_{\min}$ minimises $\langle z, \cdot \rangle$, $\langle z^*, v_{\min} \rangle - m^* = \langle z, v_{\min} - v' \rangle + \langle z^* - z, v_{\min} - v' \rangle \leq rL$. Hence $\langle z, v_i - v_{\min} \rangle = \langle z^*, v_i - v_{\min} \rangle + \langle z - z^*, v_i - v_{\min} \rangle \geq \tau_i - rL - r\|v_i - v_{\min}\|$, and the rule fires when this exceeds $r\|v_i - v_{\min}\|$, for which $\tau_i > 3rL$ suffices since $\|v_i - v_{\min}\| \leq L$. For the activation time, $r^2 \leq 2R(\text{UB} - \text{LB})$ (for $\text{LB} \geq 0$, $r^2 = (\text{UB} - \text{LB})(\text{UB} + \text{LB})$ and $\text{UB} + \text{LB} \leq 2R$; for $\text{LB} < 0$, $r^2 = \text{UB}^2$ and $\text{UB} \leq 2(\text{UB} - \text{LB})$), and $2R(\text{UB} - \text{LB}) \leq 2Rg^{\text{FW}}/\text{UB} \leq (2R/\delta^*)(2h + R\sqrt{2h}) \leq (4R^2/\delta^*)\sqrt{2h}$ (Lemma 5, with equality in the first step for the freshly evaluated bound; the last step uses $h \leq R^2/2$, which holds for every feasible iterate since $h \leq \frac{1}{2}\|z\|^2$); requiring $r^2 \leq \tau^2/(9L^2)$ yields $h_t < H$, and Theorem 3 with $\log(1/(1 - \rho)) \geq \rho$ converts it into the iteration bound (the $\max\{1, \cdot\}$ covers $h_0 \leq H$, where the condition holds from the start by monotonicity). The removal is executed at the first screening round after the condition holds; permanence follows from the safety of the rule. $\blacksquare$

Appendix B. Exact reductions

L_2 soft margin. The problem $\min \frac{1}{2}\|w\|^2 + \frac{C}{2} \sum_i \xi_i^2$ is the hard-margin problem on the augmented points $\tilde{x}_i = [x_i; e_i/\sqrt{C}]$, e_i the i -th standard basis vector, whose Gram matrix is $K + I/C$ (K the Gram matrix of the original points) and whose augmented hulls never intersect; let δ_C denote their distance. The augmentation is never materialised: the extra coordinates of an iterate are exactly its convex weights, so every augmented inner product is a cached original-space product plus a sparse diagonal correction, $\langle \tilde{p}, \tilde{v}_i \rangle = \langle p, v_i \rangle + \alpha_i/C$ and $\|\tilde{p}\|^2 = \|p\|^2 + \|\alpha\|^2/C$. The separator is recovered as $w = 2(p - q)/\delta_C^2$, $b = (\|\tilde{q}\|^2 - \|\tilde{p}\|^2)/\delta_C^2$, and the primal value is $P = 2/\delta_C^2$. Validation: the solver agrees with an exact QP on the explicitly augmented problem to 10^{-10} across random instances and values of C ; the regularisation path δ_C is monotone in C ; on a9a ($C=0.1$) its certified primal 687.07 equals LIBLINEAR’s to five digits.

ν -SVM. The L_1 (hinge) soft margin is the nearest-point problem between *reduced* convex hulls $R(V, \mu) = \{\sum a_i v_i : \sum a_i = 1, 0 \leq a_i \leq \mu\}$, $\mu = 2/(\nu\ell)$ with $\ell = n + m$ the number of training points [8]. Two changes: the extreme-point oracle becomes a capped top- k blend (weight μ on the $\lfloor 1/\mu \rfloor$ best points in the search direction, via a partial sort, still $O(n)$), and the primary step is the capped MDM transfer with blended toward steps as fallback; caching and the reduced support-function lower bound carry over unchanged. Validation: agreement with the exact box-constrained QP to 10^{-9} – 10^{-14} and with LIBSVM’s NuSVC to $\sim 10^{-10}$ through the mapping $\delta^* = \|w_{\text{svc}}\|/\varsigma$, w_{svc} LIBSVM’s weight vector and ς the per-class sum of the rescaled dual coefficients. The reduced hull is a polytope, so (1) still applies, but its vertices are blends and the capped transfer is not the pairwise step analysed in §4; the analysis of this variant is left open.

Kernels. The algorithm touches data only through inner products, so kernelisation changes plumbing, not algorithm: Gram columns become kernel columns $\kappa(V, x_i)$, squared norms become kernel diagonals, iterates exist purely as convex weights (the exact cache refresh is reconstructed from the support’s cached kernel columns), and the classifier is the kernel expansion of the witness-pair bisector. A ridge $K + I/C$ gives the kernel L_2 margin by the same diagonal trick. Validation: the linear kernel reproduces the Euclidean solver to 10^{-12} ; RBF distances match the exact kernel QP to 10^{-9} and LIBSVM’s hard-margin solution, via $\|w_{\mathcal{H}}\|^2 = (\alpha \circ y)^\top K(\alpha \circ y)$, $w_{\mathcal{H}}$ the feature-space normal, for LIBSVM’s dual coefficients α and labels y , to 10^{-5} ; kernel SMO matches the QP to 10^{-8} .

Appendix C. Experimental details

Solver settings and tolerance equivalence. ETA: block transfers with $k = 16$, guarded, away fallback, pairwise anti-zig-zag, stop at certified relative gap 10^{-3} , iteration cap 2×10^6 , 600 s cap. Tables 1, 6 and 7 were produced by the released code: the schedule of Lemma 10 and the case-(b) fallback in every path. Tables 3, 8, 2 and 5, the matched-tolerance rows of Table 4 and Figures 1 and 3 were rerun with it. The submitted version’s single-surviving-pair path took the possibly capacity-clipped MDM transfer directly, bypassing the fallback; the fix changes a block trajectory only on such iterations. Block assembly copies the k' cached columns into a $k' \times n$ array, an $O(k'n)$ copy, before the $O(k'^2)$ products; screening (every 25 iterations) only in the LIN cell, where it never activated at 10^{-3} (Theorem 6 gives a sufficient condition only, which these runs stop before meeting). Our SMO: maximal-violating-pair selection, KKT gap 10^{-3} , same caps; hard margin as

$C = 10^{12}$; kernel SMO on $K + I/C$, with a 512-row least-recently-used kernel-row cache, so its row counts and times include rows recomputed after eviction, while ETA’s column cache never evicts (a full-cache SMO rerun was not made). LIBSVM on the precomputed $K + I/C$ kernel (scikit-learn SVC, kernel=’precomputed’, $C=10^6$, tolerance 10^{-3} , five seeds; `src/libsvm_kl2.py`) solves the same kernel L_2 margin: the RBF kernel with $1/C$ on the diagonal is built with BLAS in 1–3 s and the fit takes 0.5–2.2 s, with distances equal to K-ETA’s to five digits. LIBLINEAR (scikit-learn 1.8 LinearSVC with its default dual=’auto’, which selects the primal trust-region Newton solver on these datasets, not dual coordinate descent): squared hinge with parameter $C/2$, so that its objective equals $\frac{1}{2}\|w\|^2 + \frac{C}{2} \sum \xi_i^2$; tolerance 10^{-6} , chosen so that its final primal matches ETA’s certified value on every cell where ETA converges except gisette at $C=10$ (a9a 687.07 vs 687.07; gisette 0.14900 vs 0.14893, inside ETA’s certified 8×10^{-4} ; ijcnn1 550.6 vs 550.5 at ETA gap 0.01); this tolerance is also why LIBLINEAR needs 18,000 iterations and 51 min on gisette at $C=1$, and 35,000 iterations and 1.4 h at $C=10$, where it nevertheless stops 5% above the certified optimum (0.1566 vs 0.1490). The gisette LIBLINEAR times in Table 2 are therefore the cost of matching ETA’s primal; Table 4 reports the same configuration at scikit-learn’s default tolerance 10^{-4} (five seeds, run separately without concurrent load): 3.5–51 s, but primals 5–24% above the certified optimum and lower test accuracy. Intercept scaling 100, so LIBLINEAR regularises $b/100$ where the geometric formulation leaves b free (the primals agree to five digits); iteration cap 2×10^5 . LIBSVM (scikit-learn SVC): tolerance 10^{-3} , 1000 MB cache, iteration cap 2×10^6 (reported as *maxiter* when hit); hard margins as $C = 10^6$; RBF kernel $\exp(-\|x - x'\|^2/d)$. Oracle counts: ETA counts Gram or kernel columns evaluated against a full class ($O(nd)$ or $O(n_k d)$ each); SMO counts kernel rows, the same unit. Columns are cached without eviction and the block Gram matrix Γ is assembled from four cached entries per pair, so peak cache memory is the column count times n_k doubles: 0.73 GB for the largest cell (covtype KL2, 9,100 columns), 0.61 GB for a9a KL2 and 0.36 GB for ijcnn1 KL2; measured peak resident memory in the isolated runs of Table 6 is 0.74 GB on gisette (data-dominated) and 0.49–0.50 GB on ijcnn1 KL2 (0.68 GB for BPCG).

Data and seeds. a9a, w8a, ijcnn1 and gisette use the LIBSVM training and test files; covtype (binary, scaled) is subsampled to 100,000 points and split 75/25 by seed. Features are standardised on the training set. Kernel cells use a class-balanced subsample of at most 5,000 points per class (w8a’s minority class has 1,479 points, hence $n_k = 6,479$; gisette uses all 6,000). Seeds vary the covtype split, the kernel subsamples and the timing environment; on datasets with fixed splits the deterministic solvers return identical solutions across seeds, which is why several accuracy CIs are zero. Confidence intervals are t -based 95% intervals over seeds.

Hardware and concurrency. One dedicated 32-vCPU Intel Xeon 2.6 GHz Lightning Studio, 122 GB RAM, Linux, Python 3.12, NumPy 2.4 with OpenBLAS 0.3.31, `OMP_NUM_THREADS=1`. The L_2 battery of Table 2 ran four single-threaded cells at a time; the regime battery of Table 1 ran 12 cells concurrently with the ablation of Table 6, the BPCG runs of Table 7 and the LIBSVM kernel runs, 19 processes in all (the ablation was then rerun in isolation, below); the default-tolerance rows of Table 4 were made with no concurrent load; the five trials of Figure 3 ran as seven concurrent single-threaded jobs on an otherwise idle machine and reproduce the submitted run within its intervals; the absolute times of Tables 3 and 8 are about $3\times$ those of an earlier run with identical iteration counts, so their load is uncertain and only their within-table ratios should be read. The uncapped traces of Figure 1 and Table 5 ran one process at a time. Ratios within a battery share the load but not necessarily its effect, since dense BLAS, array scans and Python set operations contend

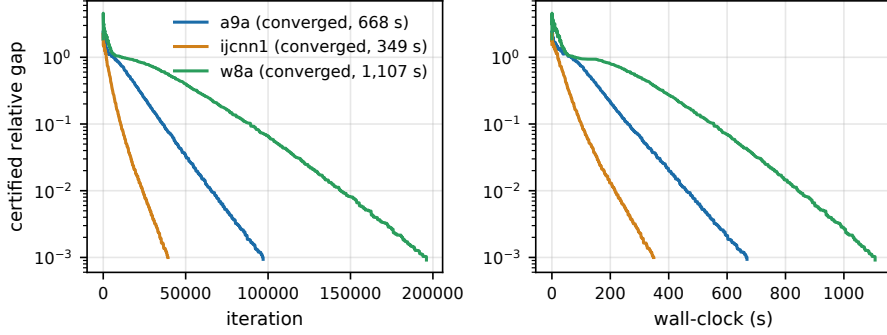


Figure 1: Certified relative gap of ETA on a9a, w8a and ijcn1 (L_2 , $C = 0.1$, uncapped, seed 0), versus iteration and wall-clock: linear-rate convergence in the dense-support regime, reaching the certified 10^{-3} in 349–1,107 s against the 600 s cap of Table 2. Heuristically (δ has no known bound in terms of the class hulls), $n_s \approx 0.6n$ active atoms forming a near-regular simplex have pyramidal width $\sqrt{2/n_s}$ times the side [3], so $\rho \sim 1/n_s$ and the cost is $O(n^2)$ -type.

for different resources, and the intervals measure the observed variation, not this systematic part; so only ratios whose intervals separate clearly should be read as results. The isolated rerun of Table 6 (`src/step_profile.py`, one configuration at a time on an otherwise idle machine) measures the effect: under the 19-process load the gisette MDM, guarded $k=1$ and $k=16$ configurations took 36.6, 27.7 and 37.8 s and ijcn1’s 16.0, 12.8 and 7.9 s; in isolation they take 30.1, 30.3, 30.8 and 15.3, 13.5, 7.0 s. The load thus manufactured one ratio ($1.3\times$ for guarded $k=1$ over MDM on gisette, a tie in isolation) while the ijcn1 ratios and the BPCG comparison (20.2 vs 5.9 s loaded, 21.1 vs 7.0 s isolated) survive.

Reference solver in the KL2 cells. The hinge-loss SVC (`rbf`, $C=1$), a different objective, took 8.6 ± 2.9 (a9a), 3.4 ± 0.3 (covtype), 83.5 ± 13.1 (gisette), 1.1 ± 0.0 (ijcn1) and 4.9 ± 1.2 s (w8a), with test accuracies 0.789, 0.783, 0.977, 0.950 and 0.975, within one point of the L_2 solutions and on either side of them.

Step ablation, blocks and shrinking (synthetic). Iteration counts at $\varepsilon = 10^{-6}$ (single instances): $d=30$, $n=200$: toward 158,941, midpoint 148,996, away 106, pairwise 181; $d=100$, $n=1,000$: toward and midpoint fail within 300,000, away 15,886, pairwise 3,921. At the working tolerance 10^{-3} the pairwise/away gain over the midpoint heuristic is only $\sim 11\%$, which is why the heuristic sufficed in [2]. Block transfers cut iterations $3\text{--}25\times$ relative to single MDM (e.g. $43,486 \rightarrow 3,001$ at $k=16$ on a dense-support L_2 instance at $d=1,000$) and are the only configuration that reaches the certified tolerance on MNIST odd-vs-even. Figure 2 shows the screening speed-ups and Figure 3 the consolidated comparison, five trials per class (mean $\pm$ 95% CI): on the four hard-margin classes the optimised solver is $1.1\text{--}1.2\times$ faster than LIBSVM on average, with overlapping intervals (0.63 ± 0.04 vs 0.75 ± 0.15 s at $d=1,000$; 17.9 ± 1.6 vs 20.6 ± 3.7 s at $d=10,000$; 0.72 ± 0.13 vs 0.80 ± 0.18 s at $\varepsilon=10^{-5}$, where the paper-style configuration fails to converge in every trial; RBF 2.33 ± 0.06 vs 2.50 ± 0.12 s); on the synthetic L_2 margin ($d=1,000$, $C=1$) it ties LIBLINEAR (20.7 ± 9.7 vs

Table 2: L_2 soft-margin battery: five seeds, 600 s per solver for ETA and our SMO (our own implementation; LIBSVM does not solve this objective), LIBLINEAR at tolerance 10^{-6} (Appendix C) with no cap; released code, four single-threaded cells at a time on the 32-vCPU machine; mean $\pm$ 95% CI, which on the fixed-split datasets (all but covtype) measures timing noise only. “cap” = timed out in every seed; ETA’s certified relative gap is reported at its stopping point (mean [min, max]); accuracies are those of the returned iterate.

dataset ($n + m / d$)	C	ETA (s)	SMO (s)	LIBLINEAR (s)	ETA gap at stop	acc. ETA / SMO / LIBLINEAR
gisette (6,000 / 5,000)	0.1	30.0 ± 3.6	212 ± 26	462 ± 70	9×10^{-4}	.981 / .981 / .981
	1	29.6 ± 3.0	212 ± 24	$3,060 \pm 491$	10×10^{-4}	.981 / .981 / .981
	10	28.3 ± 2.9	209 ± 26	$5,031 \pm 337$	10×10^{-4}	.981 / .981 / .977
a9a (32,561 / 123)	0.1	cap	cap	2.6 ± 0.6	0.0045 [0.0037, 0.0060]	.850 / .850 / .849
	1	cap	cap	3.2 ± 0.9	0.78 [0.75, 0.83]	.823 / .801 / .849
	10	cap	cap	1.4 ± 0.4	1.2 [1.2, 1.2]	.505 / .768 / .849
ijcnn1 (49,990 / 22)	0.1	418 ± 23	200 ± 14	0.2 ± 0.0	10×10^{-4}	.918 / .918 / .918
	1	cap	cap	0.2 ± 0.1	0.29 [0.21, 0.36]	.923 / .918 / .918
	10	cap	cap	0.1 ± 0.0	1.2 [1.2, 1.2]	.601 / .900 / .918
w8a (49,749 / 300)	0.1	cap	cap	14.2 ± 3.5	0.16 [0.08, 0.24]	.987 / .986 / .987
	1	cap	cap	11.9 ± 0.7	0.92 [0.92, 0.93]	.895 / .978 / .987
	10	cap	cap	14.6 ± 0.9	1.6 [1.5, 1.7]	.967 / .972 / .987
covtype (100,000 / 54)	0.1	cap	cap	22.5 ± 21.2	1.0 [1.0, 1.0]	.572 / .701 / .756
	1	cap	cap	4.3 ± 2.3	1.1 [1.0, 1.1]	.537 / .545 / .756
	10	cap	cap	3.7 ± 1.1	1.3 [1.3, 1.6]	.502 / .515 / .756

19.4 ± 4.4 s); on MNIST odd-vs-even it is $2.7\times$ slower (172 ± 20 vs 63 ± 22 s; single MDM never converges) and on the ν -SVM $1.7\times$ slower (61.0 ± 5.0 vs NuSVC’s 36.5 ± 5.6 s). Distances agree within tolerance on every row. An earlier run of the same script on a shared development container had shown the optimised solver 1.6 – $2.5\times$ ahead on six of the seven classes; that machine’s Python-to-C timing ratio did not reproduce on dedicated hardware, and only the dedicated numbers are reported here.

Table 3: Two-sided schedules on the synthetic instances of this appendix ($n=5,000$ per class, $k=16$): the unconditional V -then- W order used for the batteries versus the schedule of Lemma 10 (larger-gap side first, second side skipped after a drop). Drop steps and skipped second-side steps are counted over the run.

instance	schedule	time (s)	iterations	drops	skips	distance
hard margin, $d=1000$, $\varepsilon=10^{-5}$	unconditional	0.8	16	0	0	55.339365
hard margin, $d=1000$, $\varepsilon=10^{-5}$	Lemma 10	0.8	16	0	0	55.339365
L_2 overlap, $d=100$, $C=1$	unconditional	25.2	12,311	785	0	0.078400
L_2 overlap, $d=100$, $C=1$	Lemma 10	23.3	10,701	599	316	0.078400
L_2 overlap, $d=1000$, $C=1$	unconditional	12.9	2,901	182	0	0.578143
L_2 overlap, $d=1000$, $C=1$	Lemma 10	13.4	3,001	180	121	0.578143

The guarded single pair already halves the iterations of plain MDM on ijcnn1 ($1.4\times$ on gisette): the drop steps it takes (on 0.2 – 2% of its iterations) are what plain pairwise FW replaces by capacity-clipped swaps. Blocks then cut iterations a further 4.2 – $4.4\times$ ($k=4$, run only under load: 1,071 and

Table 4: LIBLINEAR on gisette at its default tolerance (10^{-4}) versus the matched tolerance (10^{-6}) of Table 2; five seeds. ETA’s certified primal is 0.14835 / 0.14893 / 0.14899 at $C = 0.1 / 1 / 10$, reached in 28–30 s with test accuracy 0.981.

C	tol	time (s)	iterations	primal (vs certified)	accuracy
0.1	10^{-4}	51 ± 1	363	0.15648 (+5.5%)	.978
0.1	10^{-6}	462 ± 70	2,434	0.14835 (+0.0%)	.981
1	10^{-4}	4.3 ± 0.1	10	0.18427 (+24%)	.974
1	10^{-6}	$3,060 \pm 491$	18,063	0.14900 (+0.05%)	.981
10	10^{-4}	3.5 ± 0.1	8	0.18487 (+24%)	.973
10	10^{-6}	$5,031 \pm 337$	35,411	0.15663 (+5.1%)	.977

Table 5: Time to 1% of the optimum, seed-0 traces. The ETA columns measure *dual* progress and LIBLINEAR’s column *primal* progress, so this is not a matched primal comparison (the primal of ETA’s returned (w, b) is not recorded in the traces). For ETA the certified dual value $2/\text{UB}_t^2$ increases to the optimum, so $s_t = 1 - (2/\text{UB}_t^2)/P^*$ with P^* the best known primal (the lower of LIBLINEAR’s 10^{-6} value and ETA’s certified value); “certified 1%” is the first time the certificate itself proves $1 - \text{LB}_t^2/\text{UB}_t^2 \leq 1\%$. ETA’s a9a, w8a and ijcn1 times are from the uncapped runs of Figure 1 (`src/time_to_tol.py`). The LIBLINEAR entries are the five-seed mean times of the matched 10^{-6} run of Table 2, an upper bound on its time to 1% (a finer tolerance sweep would give the exact value); “none” means that neither 10^{-4} nor 10^{-6} reaches 1%. On the remaining cells ETA is not within 1% at the 600 s cap ($s = 11$ –99%).

dataset	C	P^*	ETA 1%	ETA 0.1%	cert. 1%	cert. 10^{-3}	LIBLINEAR 1%
gisette	0.1	0.14835	30 s	31 s	31 s	32 s	≤ 462 s (tol 10^{-6})
gisette	1	0.14893	30 s	31 s	32 s	33 s	$\leq 3,060$ s (tol 10^{-6})
gisette	10	0.14899	24 s	26 s	26 s	27 s	none at 10^{-4} or 10^{-6}
a9a	0.1	687.07	274 s	385 s	524 s	668 s	≤ 2.6 s (tol 10^{-6})
w8a	0.1	108.80	589 s	764 s	929 s	1,107 s	≤ 14 s (tol 10^{-6})
ijcn1	0.1	550.61	108 s	172 s	256 s	349 s	≤ 0.2 s (tol 10^{-6})

2,358 iterations, times within CI of $k=16$). Away fallbacks that drop an index occur on 0.2–36% of guarded iterations, and their share grows with k : larger blocks add more indices per iteration, and the drops remove them. The time split says where fewer iterations buy time. On gisette, 86–92% of every configuration’s time is Gram-column computation ($O(nd)$ per new receiver, $d=5,000$), so MDM, guarded $k=1$, block $k=16$ and BPCG all tie at 30 s. On ijcn1 the columns cost about 1 s and the rest is per-iteration $O(n)$ work: the score update and line search inside the step, the certificate in the remainder, and least of all the scan itself. That work falls with the iteration count, though not in proportion, because a block step updates the scores with up to k' columns; the net effect is $2.2\times$ over MDM and $3.0\times$ over BPCG.

Why blocks help less on real data, and choosing k . For uncapped pairs $S = \sum_j b_j^2$, so $\hat{\eta} = (D/S - 1)/(k' - 1)$ is the effective pair correlation of Proposition 4, and the guard computes it at no cost. Instrumented seed-0 runs on the five real cells of Table 1 (gisette L_2 and the other four KL2

Table 6: Step ablation on real data with a time split, isolated runs (`src/step_profile.py`: three seeds, each configuration alone in its own process on an otherwise idle machine): the gisette L_2 margin ($C=1$, $d=5,000$) and the ijcn1 kernel L_2 cell of Table 1 ($d=22$, $n_k=9,853$). MDM is plain pairwise FW (single transfers, no guard, no fallback); guarded $k=1$ is Algorithm 1 with a single pair (guard and case-(b) fallback active); BPCG as in Table 7. Split of the ETA time: scan = the score scan and pair selection; col = Gram or kernel column computation; step = the step itself (block assembly, guard, line search, cache and score update); rest = certificate and bounds. Drops = away fallbacks that removed a support index. Every run converged to the certified gap with identical distance and accuracy per dataset. Peak resident memory: 0.74 GB for every gisette run and 0.49–0.50 GB (ETA) / 0.68 GB (BPCG) on ijcn1.

instance	step	k	time (s)	scan	col	step	rest	iterations	columns	drops
gisette L_2	MDM	1	30.1 ± 0.9	0.3	26.1	1.5	2.1	4,946	1,300	0
gisette L_2	guarded	1	30.3 ± 3.2	0.2	26.2	2.4	1.4	3,471	1,334	82
gisette L_2	block	16	30.8 ± 5.4	0.1	28.2	2.1	0.4	831	1,480	301
gisette L_2	BPCG	—	29.9 ± 2.2					10,470	1,322	87
ijcn1 kernel L_2	MDM	1	15.3 ± 2.1	1.4	1.1	7.5	5.2	12,796	4,520	0
ijcn1 kernel L_2	guarded	1	13.5 ± 0.4	0.9	0.8	8.0	3.8	6,118	4,514	14
ijcn1 kernel L_2	block	16	7.0 ± 0.6	0.3	0.9	4.8	1.0	1,396	4,590	98
ijcn1 kernel L_2	BPCG	—	21.1 ± 0.9					18,034	4,512	19

cells; `src/block_diagnostics.py`, one process at a time, $k \in \{1, 4, 16, 32\}$) give median $\hat{\eta} = 0.07$ (gisette), 0.11 (a9a), 0.17 (w8a), 0.26 (ijcn1) and 0.29 (covtype). Proposition 4 then guarantees a 16-pair block of equal pair gains only 3–8 transfers’ worth, against 16 for orthogonal pairs, and drop steps take a further share (36% of gisette’s iterations). On the synthetic instances the iteration cut grows with d ($6.8\times$ at $d=100$, $14.5\times$ at $d=1,000$, Table 8), as expected if random difference directions become more orthogonal in higher dimension; we did not measure $\hat{\eta}$ there. If an iteration costs $a + bk$ (scan plus per-pair work) and a block is worth $k/(1 + \eta(k - 1))$ transfers, the cost per transfer is minimised at $k^* = \sqrt{(a/b)(1 - \eta)/\eta}$; with a and b fitted per cell and $\hat{\eta}$ for η this gives $k^* = 7\text{--}14$, to be capped by the support size and the column cache. The measured best was $k=4$ on gisette and ijcn1 and $k=16$ on the other three cells, within $1.3\times$ of each other; on gisette, where columns take 93% of the time, no k helps.

BPCG here is the blended pairwise step of [12] on the atoms $v_i - w_j$ of Z with the separable LMO of Lemma 7, exact line searches and the certificate of Lemma 5; its scores are maintained from the same cached columns, so column counts are comparable. It needs 12–35 $\times$ more iterations than block ETA on every cell, each an $O(n + m)$ scan; the block’s fewer scans buy 2.5–5.6 $\times$ in wall-clock where scans dominate (KL2) and nothing where columns do (gisette L_2).

Appendix D. Extended related work

Aggregated directions. Boosted FW [13] builds its direction by greedily aligning a combination of atoms with the negative gradient, at several oracle calls per iteration, and inherits the FW rate through an alignment condition; our block fixes the pair coefficients γ_j from one scan and relies on the guard, not on alignment, for its rate.

Table 7: BPCG [12] versus block ETA ($k=16$) on the same objective with the same certificate ($\varepsilon=10^{-3}$), the same column cache and the same 600 s cap (`src/bpcg.py`); three seeds, run concurrently with Table 1; the isolated rerun of Table 6 gives `ijcnn1 KL2` 21.1 ± 0.9 vs 7.0 ± 0.6 s and `gisette  $L_2$ ,  $C=1$` 29.9 ± 2.2 vs 30.8 ± 5.4 s. Every cell converged; distance and test accuracy agree per cell to five digits. Columns are kernel or Gram columns computed; drops count BPCG’s capacity-clipped local steps and ETA’s away drops.

cell	solver	time (s)	iterations	columns	drops
<code>gisette L_2, $C=1$</code>	BPCG	34.2 ± 20.4	10,470	1,322	87
<code>gisette L_2, $C=1$</code>	ETA	34.3 ± 15.0	831	1,480	301
<code>gisette KL2</code>	BPCG	213 ± 28	8,702	3,641	3
<code>gisette KL2</code>	ETA	83.9 ± 19.0	251	3,653	4
<code>w8a KL2</code>	BPCG	22.3 ± 5.8	12,342	3,577	12
<code>w8a KL2</code>	ETA	5.0 ± 2.2	603	3,591	50
<code>a9a KL2</code>	BPCG	65.4 ± 5.7	28,653	7,605	24
<code>a9a KL2</code>	ETA	11.7 ± 2.2	1,489	7,616	35
<code>covtype KL2</code>	BPCG	90.6 ± 11.2	38,422	9,065	15
<code>covtype KL2</code>	ETA	20.3 ± 1.1	3,336	9,074	26
<code>ijcnn1 KL2</code>	BPCG	20.2 ± 1.6	18,034	4,512	19
<code>ijcnn1 KL2</code>	ETA	5.9 ± 1.1	1,396	4,590	98

Table 8: Block-size ablation on the synthetic instances (current code): single MDM ($k=1$) versus blocks of k transfers. Iterations fall with k ; wall-clock does not fall in proportion, because each block iteration touches more columns.

instance	k	time (s)	iterations	columns	drops
hard margin, $d=1000$, $\varepsilon=10^{-5}$	1	0.6	301	104	0
hard margin, $d=1000$, $\varepsilon=10^{-5}$	4	0.6	36	103	0
hard margin, $d=1000$, $\varepsilon=10^{-5}$	16	0.5	16	103	0
hard margin, $d=1000$, $\varepsilon=10^{-5}$	32	0.6	16	109	0
L_2 overlap, $d=100$, $C=1$	1	26.3	72,546	1,345	0
L_2 overlap, $d=100$, $C=1$	4	27.2	24,606	1,632	416
L_2 overlap, $d=100$, $C=1$	16	23.9	10,701	2,404	599
L_2 overlap, $d=100$, $C=1$	32	27.4	7,606	3,076	305
L_2 overlap, $d=1000$, $C=1$	1	24.8	43,486	1,242	0
L_2 overlap, $d=1000$, $C=1$	4	17.7	10,071	1,330	177
L_2 overlap, $d=1000$, $C=1$	16	13.5	3,001	1,348	180
L_2 overlap, $d=1000$, $C=1$	32	13.1	1,741	1,396	170

Swap steps. Blended pairwise conditional gradients [12] also remove pairwise FW’s swap-step bound with a guard, there between the pairwise step and a FW step, and [14] avoid bad steps with a short-step chain. Our guard compares an aggregated block of k transfers with the single MDM step and falls back to an away step, so the drop-step count of away-step FW applies unchanged. Block-coordinate FW [9] blocks over *coordinates* of a product domain rather than aggregating pairs on one polytope.

Screening and identification. Gap-safe rules [15, 17, 18] screen with a radius from the duality gap and identify the support in finite time; the active-set complexity of away-step FW [19] and

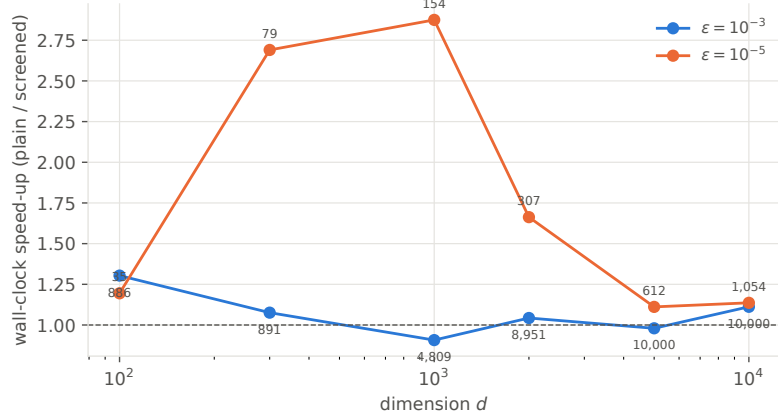


Figure 2: Safe screening: wall-clock speed-up vs dimension at two tolerances on the separated synthetic protocol; labels are surviving points of 10,000. Made with the toward-step solver of the development phase (`src/shrink_benchmark.py`); the rule and its activation do not depend on the step.

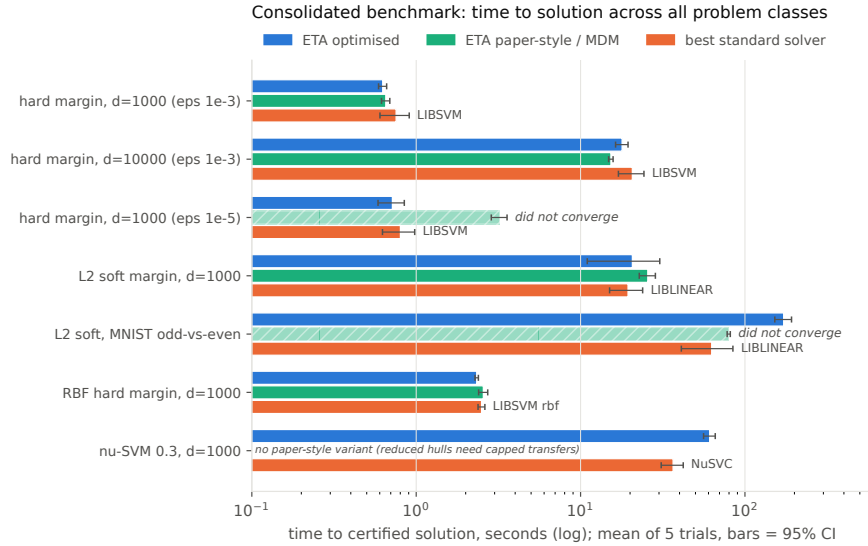


Figure 3: Time to certified solution on seven problem classes: optimised ETA vs the paper-style configuration vs the best standard solver per class; five trials per class, mean with 95% CI. The ν -SVM row has no paper-style bar because the toward-step algorithm of [2] has no reduced-hull variant (reduced hulls need capped transfers).

the strict-complementarity bounds of [20] give the same identification from the algorithm's side.

Theorem 6 is the polytope-distance instance, with constants tied to the TA certificate; the SVM rules of [10, 11] and the exact data reduction of [16] are its ancestors.

Relation to other FW variants. The toward, away, pairwise and block rows of the step ablation in Appendix C are Frank–Wolfe, away-step FW, pairwise FW and our block step on the same polytope and objective; blended pairwise conditional gradients (BPCG) [12] is compared on real data in Table 7; Wolfe’s minimum-norm-point method [26] was not run. BPCG’s blending rule compares the away and FW gaps and its linear rate has the same pyramidal-width form as Theorem 3; our case (b) instead drops when the away step is clipped and otherwise compares the realised gains of the block, MDM, away and toward candidates, so each non-drop step is at least as good as the best single candidate, and it is the away fallback rather than the blending that yields the drop-step count. The facial distance of [27] is a computable alternative to the pyramidal width; a bound on δ for the difference atom set $A = \{v_i - w_j\}$ in terms of the two class hulls is open.

Synthetic results in brief. Away and pairwise steps converge at $\varepsilon = 10^{-6}$ where the toward and midpoint steps of [2] do not. Screening reduces 10,000 points to 36–1,054 survivors at $\varepsilon = 10^{-5}$. On seven synthetic and small-real problem classes (Figure 3, five trials each) the optimised solver is 1.1–1.2 $\times$ faster than LIBSVM on the four hard-margin classes (intervals overlap), ties LIBLINEAR on the synthetic L_2 margin, and is 1.7–2.7 $\times$ slower than NuSVC and LIBLINEAR on the ν -SVM and MNIST classes. On the four classes where single MDM also converges, blocks and guard change wall-clock by 0.9–1.2 $\times$; their clear wins are the two classes (tight tolerance, MNIST) where single MDM does not converge at all.

Appendix E. Numerical verification of the analysis

Lemma 1 was checked on 4,000 random ensembles including heavily clipped and boundary cases (worst slack at machine precision); Proposition 4 on 500 near-orthogonal ensembles; Lemma 5 to 10^{-9} at arbitrary iterates; Lemma 10 on instrumented two-sided runs (no drop iteration added a support index, and every non-drop iteration met the gain bound with M replaced by the computable upper bound $\text{diam } V + \text{diam } W$); and Theorem 6 on instances whose optimal face was computed by an exact QP: at every activating round all zero-weight non-face points were screened, and the final working set equalled the optimal face exactly. Over 5,000 instrumented live block steps none increased the objective. All checks are test files in the released code.